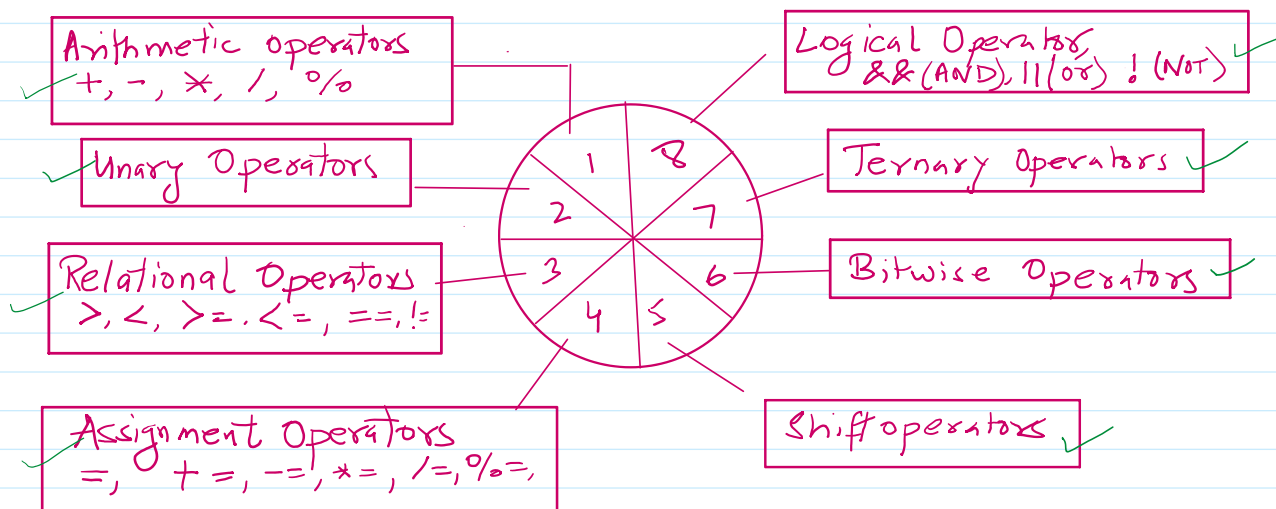


Java Operators

26 May 2026 20:18

Operators are an essential part of any programming language. In Java, operator is a symbol that is used to perform operations.



Java Unary Operator

The Java unary operators requires only one operand. Unary operators are used to perform various operations, that is, incrementing or decrementing a value by 1. Negating an expression and inverting the value of a boolean. Observe the following table.

Operator	Name	Description
+	Unary Plus	Unary plus operator indicates positive value (numbers are positive without this, however).
-	Unary minus	The unary minus operator can be used to make a positive value negative.
!	Logical Complement Operator	The NOT operator is used to convert a boolean value. In other words, it is used to reverse the logical state of an operand.
++	Increment	It is used to increase the value of an operand by one. It can be done in two possible ways. One is post increment (Operand++) and the other is pre-increment(++Operand). In post increment, the value of the operand is used first, then its value is increased by 1. In pre-increment, the value of the operand is incremented first by one, then the incremented value of the operand is used. For example: <pre>int x = 5; x 5 (int) x++; x 6 (int) ++x; x 7</pre> old value x 5 D&P - increment

```

0 (int)
++x;
x
7 (int)
int y = x++ + ++x;
y
16 (int)

y = 16 9 18
y = y++ + x++ + ++y;
y
43 (int)

```

Handwritten notes for the first example:
 - "old value" points to the 7 in `x`.
 - "pre-increment" points to the `++x`.
 - "9" points to the value of `x` after pre-increment.
 - "18" points to the value of `++x`.
 - "16" points to the value of `y` after the first addition.
 - "43" points to the final value of `y` after the second addition.

--

Decrement

It is used to decrease the value of an operand by one. It can be done in two possible ways. One is post-decrement (`operand--`) and other is pre-decrement (`--operand`).

In post-decrement, the value of the operand is used first and then its value is decreased by 1. In pre-increment, the value of the operand is decremented first by one, then the decremented value of the operand is used.

For example:

```

x
10 (int)
x--;
x
9 (int)
--x;
x
8 (int)
y = x-- - --x;
y
2 (int)
y = y-- + ++x - ++y;
y
7 (int)

```

Handwritten notes for the decrement example:
 - "8" points to the value of `x` after pre-decrement.
 - "6" points to the value of `--x`.
 - "2" points to the result of `8 - 6`.
 - "2" points to the value of `y` after the first calculation.
 - "7" points to the final value of `y` after the second calculation.

Ternary Operator: Java ternary operator is used as one line replacement for `if-then-else` statement and used a lot in Java programming. It is the only conditional operator which takes three operands.

Variable = Expression1? Expression2 : Expression3;

Handwritten annotations:
 - "true" points to Expression2.
 - "false" points to Expression3.
 - "resultant variable" points to Variable.
 - "logical expression" points to Expression1.
 - "logical or values" points to Expression2 and Expression3.

For example:

```
String z = (x == y)? x+ " and " + y + " is equal" : x+ " and " + y + " is not equal";
```

```
z
"7 and 7 is equal"    (String)
```

```
int max = (x > m)? x: m;
```

```
max
10    (int)
```

```
int a = 5;
```

```
a
5    (int)
```

```
int b = 10;
```

```
b
10   (int)
```

```
int c = 8;
```

```
c
8    (int)
```

```
max = (a > b && a > c)? a : (b>c)? b : c;
```

```
max
10   (int)
```

```
c = 15
```

```
15   (int)
```

```
a
5    (int)
```

```
b
10   (int)
```

```
max = (a > b && a > c)? a : (b>c)? b : c;
```

```
max
15   (int)
```

Bitwise operators: Java bitwise operators are used to perform the manipulation of individual bits of A number, and with any of the integer types.

Operator	Operator Name	Description
	Bitwise OR	It is a binary operator which gives OR of the input values bit by bit.
&	Bitwise AND	It is a binary operator which gives AND of the input values bit by bit.
^	Bitwise XOR	It is a binary operator which gives XOR of the input values bit by bit.
~	Bitwise Complement	The bitwise complement operator is also a unary operator. It makes every bit 0 to 1 and 1 to 0.

For example:

```
byte f = 5;
```

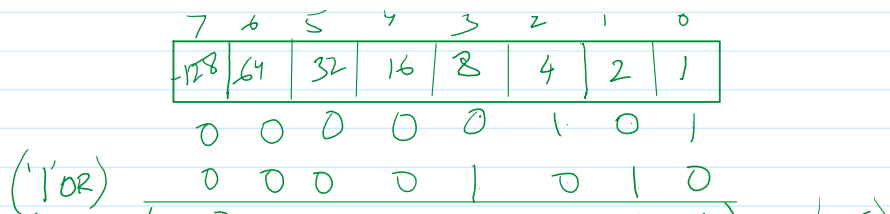
```
byte g = 10;
```

```
f
```

```
5    (byte)
```

```
g
```

```
10   (byte)
```



g
10 (byte)

$$\begin{array}{r} \text{('OR)} \\ \begin{array}{cccccccc} 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{array} = (15) \end{array}$$

OR(1)

odd number of 1's in a row outputs 1.

m	n	m n	m & n	~m	~n	m ^ n
0	0	0	0	1	1	0
0	1	1	0	1	0	1
1	0	1	0	0	1	1
1	1	1	1	0	0	0

$$\begin{array}{r} \text{('AND)} \\ \begin{array}{cccccccc} 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ -128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} = 0 \end{array}$$

$$\begin{array}{r} \begin{array}{cccccccc} 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ -128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \end{array} \\ f = (0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1) \\ (\sim f) \quad 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 0 \\ -128 + 64 + 32 + 16 + 8 + 2 = -6 \\ -128 + 122 \end{array}$$

$$\begin{array}{r} \text{('AND)} \\ \begin{array}{cccccccc} 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ -128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{array} = 15 \end{array}$$

$$\begin{array}{r} \text{('AND)} \\ \begin{array}{cccccccc} 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ -128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \leftarrow 6 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \leftarrow 10 \\ \hline 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \leftarrow 12 \end{array} \end{array}$$

byte f = 5;
byte g = 10;
f
5 (byte)
a

```

f
5 (byte)
g
10 (byte)
byte r = f|g;
Error: incompatible types: possible lossy conversion from int to byte
y = f|g;
y
15 (int)
y = f&g;
y
0 (int)
y = ~f;
y
-6 (int)
y = f^g;
y
15 (int)
y = 6^g;
y
12 (int)

```

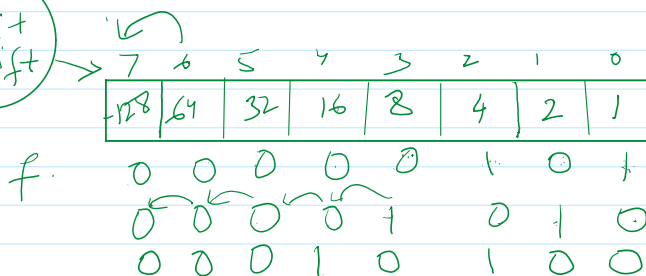
The output of operation always generates 'int' values.

↑
lower data type
than 'int' so we have
this error.

Shift operators: Java shift operator works on the bits of the data. It shifts the bits of the number from left to right or right to left. The following table shows the type of shift operator in Java.

Operator	Operator Name	Description
<<	Left Shift Operator(signed)	All of the bits shift to left by a given number using this operator.
>>	Right Shift Operator(signed)	All of the bits shift to right by a given number using this operator.
>>>	Unsigned Right shift operator	It is similar to the right shift operator (signed). However, the vacant left most position is occupied with zero in place of the signed bit.

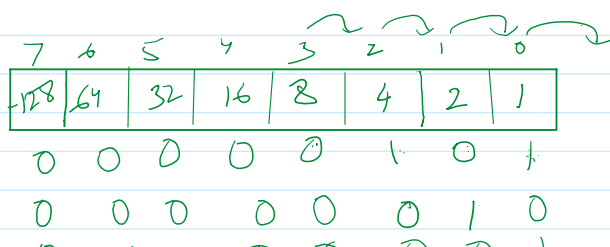
Left
shift



$$y = f \ll 2$$

$$= 20$$

Right
shift



$$y = f \gg 2$$

$$= 10$$

```

0 0 0 0 0 1 0 1
0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1

```

= 1

7	6	5	4	3	2	1	0
-128	64	32	16	8	4	2	1

```

0 0 0 0 0 1 0 1
1 1 1 1 1 0 1 0

```

= d >> 2

- 6

- 5

d =

1	1	1	1	1	0	1	1
1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	0

-128 64 32 16 8 4 2

-128 + 126 = -2

[if sign bit is '1', while shifting it will shift '1']

```

f
5 (byte)
g
10 (byte)
y = f << 2;
y
20 (int)
y = f >> 2;
y
1 (int)
byte d = -5;
y = d >> 2;
y
-2 (int)

```

```

String bin = Integer.toBinaryString(-5);
bin
"1111111111111111111111111111011" (String)
bin = Integer.toBinaryString(-5>>2);
bin
"111111111111111111111111111110" (String)
f
5 (byte)
bin = Integer.toBinaryString(-5>>>2);
bin
"111111111111111111111111111110" (String)

```